

Flash Sale — Gestion de Stock

Réservation courte + Anti-survente (Redis NoSQL)

Module : Bases de Données NoSQL

SGBD imposé : Redis (Base clé-valeur)

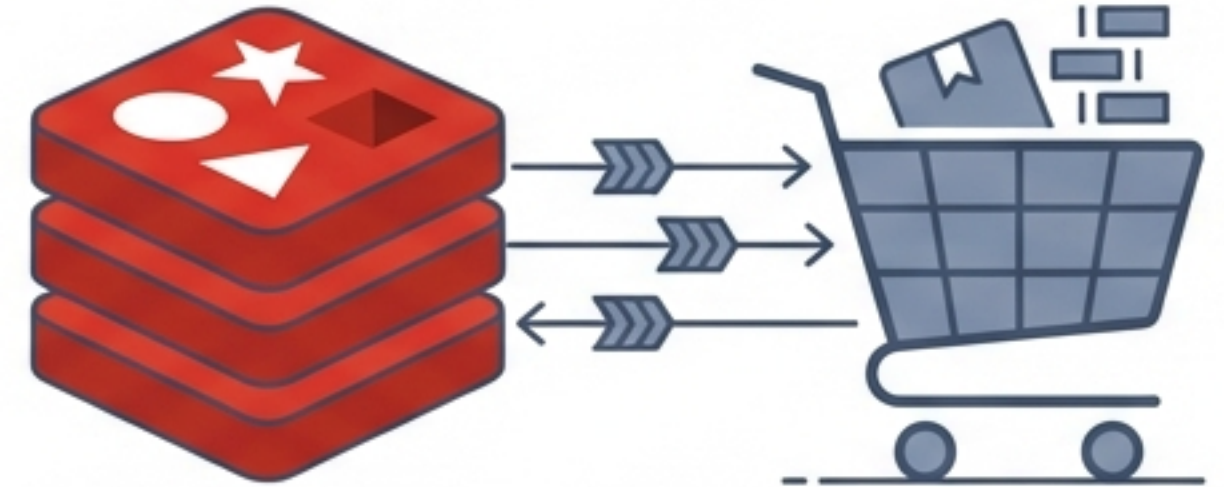
Étudiant (Chef de projet) : Khalid Morjan

Co-équipier : Anas Lahraoui

Filière : Systèmes d'Information & Intelligence Artificielle

Année : 2025–2026

Date : 5 février 2026



1. Résumé Exécutif

Ce projet répond au besoin d'une marque effectuant des ventes flash (durée minutes/heures) où le risque majeur est la survente due aux accès concurrents.

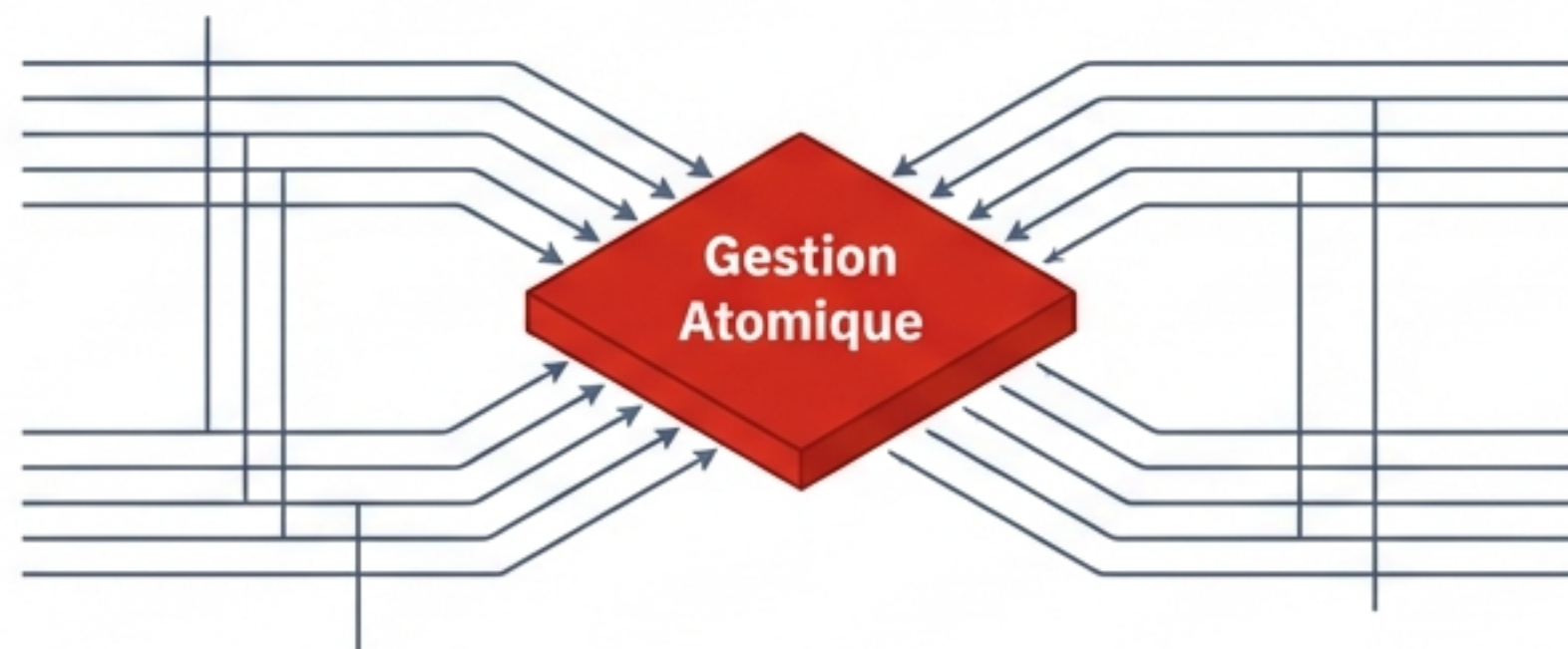
- Une réservation temporaire avec TTL (expiration automatique).
- Une atomicité (script Lua) pour empêcher les conditions de course.
- Des mécanismes de libération du stock après expiration (sweeper).
- L'idempotence contre le double paiement.
- Des métriques et un audit pour prouver la cohérence.

Chaos vs Ordre

AVANT



APRÈS



2. Contexte & Objectifs

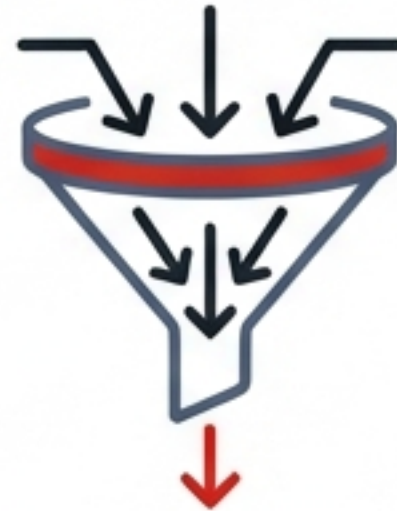
2.1 Contexte

Pendant une vente flash, de nombreux clients tentent d'acheter d'acheter simultanément. Sans mécanismes robustes, deux clients peuvent réserver le même stock.



2.2 Objectifs

- Empêcher la survente sous forte concurrence.
- Réserver temporairement un stock au passage en caisse (panier) avec expiration.
- Confirmer à l'achat et libérer automatiquement en cas d'abandon.



2.3 Contraintes

- Redis obligatoire.
- Prévention des incohérences : atomicité.
- Simulation de charge multi-clients concurrents.



3. Méthodologie Agile / Scrum

Work	Werk
Sprints	Sprints
☐ ☒ <u>FSR-1 Epic 1 - Conception & règles métier</u> DONE	☐ ☒ FSR-27 Gestion de l'expiration et libération autom... DONE
☐ ☒ FSR-11 Justification du choix Redis DONE	☐ ☒ FSR-26 Implementation de la réservation tempora... DONE
☐ ☒ FSR-18 Analyse du problème métier (flash sale & s... DONE	☐ ☒ <u>FSR-49 Epic 3 - Confirmation paiement & Indicateurs</u> DONE
☐ ☒ FSR-14 Conception du workflow global DOING	☐ ☒ FSR-58 Confirmation paiement idempotente DONE
☐ ☒ FSR-48 Définition du modèle de données Redis pour... DONE	☐ ☒ FSR-51 indicatoure de performance (KPI) DONE
☐ ☒ FSR-15 Identification des risques et cas limites DONE	☐ ☒ FSR-52 Audit des opérations DONE
☐ ☒ FSR-15 Définition des règles métier DONE	☐ ☒ FSR-52 Test du double paiement DONE
☐ ☒ <u>FSR-23 Epic 2 - Réservation et Expiration du Stock</u> DONE	☐ ☒ <u>FSR-54 Epic 4 - Simulation de charge & preuve anti-survente</u> DONE
☐ ☒ FSR-29 Documentation du Sprint 2 et validation d... DONE	☐ ☒ FSR-63 Simulation multi-clients DONE
☐ ☒ FSR-24 Conception du modèle de clés Redis pour... DONE	☐ ☒ FSR-64 Comparioson SAFE vs UNSAFE DONE
☐ ☒ FSR-25 Définition des règles d'expiration des rése... DONE	☐ ☒ FSR-65 Analyse des résultats DONE
☐ ☒ FSR-25 Simulation de plusieurs clients réservant a... DONE	☐ ☒ FSR-66 Rapport & preuves DONE
☐ ☒ FSR-27 Gestion de l'expiration et libération autom... DONE	☐ ☒ <u>FSR-67 Epic 5 - Métriques, logs et exploitation dans le rapport</u>
☐ ☒ FSR-26 Implémentation de la réservation tempora... DONE	☐ ☒ FSR-66 Compteurs système (Metrics) DONE
☐ ☒ <u>FSR-49 Epic 3 - Confirmation paiement & Indicateurs</u> DONE	☐ ☒ FSR-69 Journal d'audit DONE
☐ ☒ FSR-58 Confirmation paiement idempotente DONE	☐ ☒ FSR-78 Explosion des logs dans le rapport DONE
☐ ☒ FSR-51 Indicateurs de performance (KPI) DONE	☐ ☒ FSR-71 Détection d'anomalies DONE

Organisation

- Durée : 4 semaines (1 semaine par sprint).
- Rôles : Khalid Morjan (Product Owner / Chef de projet), Anas Lahraoui (Développeur).

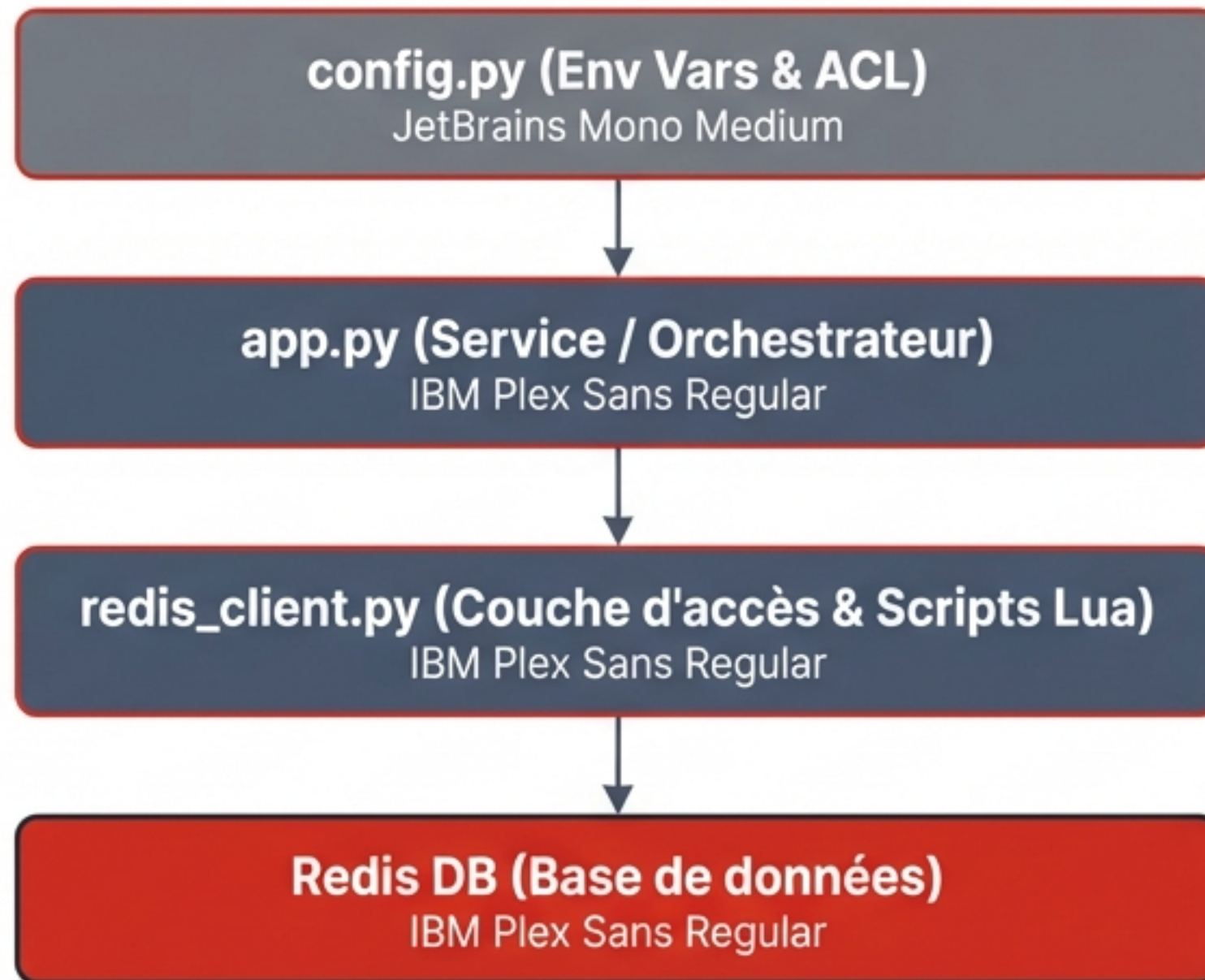
****Backlog (Extraits)****

E1: Modélisation du stock
E2: Réservation temporaire
E3: Confirmation paiement
E4: Simulation de charge

Figure 1 – Backlog Jira du projet montrant les Epics (E1–E5). (IBM Plex Sans)

4. Architecture de la Solution

4.1 Vue en couches



Composants

- **config.py**: Centralise la configuration (host/port/db, ACL, TTL).
- **redis_client.py**: Couche d'accès Redis (clés, scripts Lua, opérations atomiques).
- **app.py**: Interface CLI/service qui orchestre les cas d'usage.

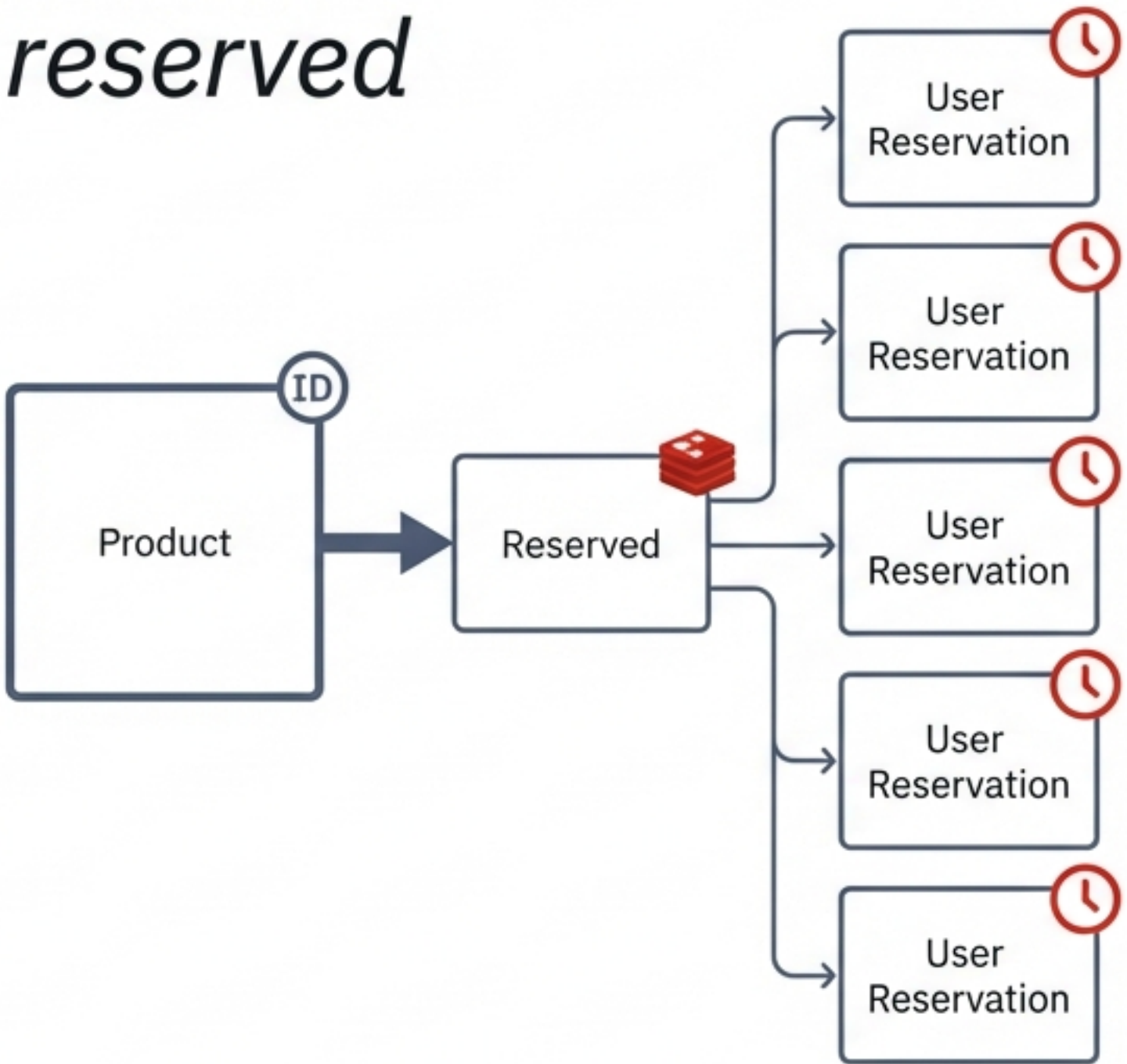
4.3 Choix techniques

- **TTL**: Pour expirer les réservations.
- **Lua**: Pour garantir l'atomicité.
- **Structures**: String/Set/Hash/List.

5. Modèle de Données (Clés & Structures)

available = stock_total – stock_reserved

Clé Pattern	Type Redis	Rôle
stock:product:<id>	String	Stock total (capacité)
stock:reserved:<id>	String	Compteur du stock réservé
reservation:<id>:<user>	String + TTL	Quantité réservée par user
active:reservations	Set	Index des réservations actives
payment:processed:<order>	String + NX	Idempotence



6. Implémentation : Initialisation & Réserveation

6.1 Initialisation

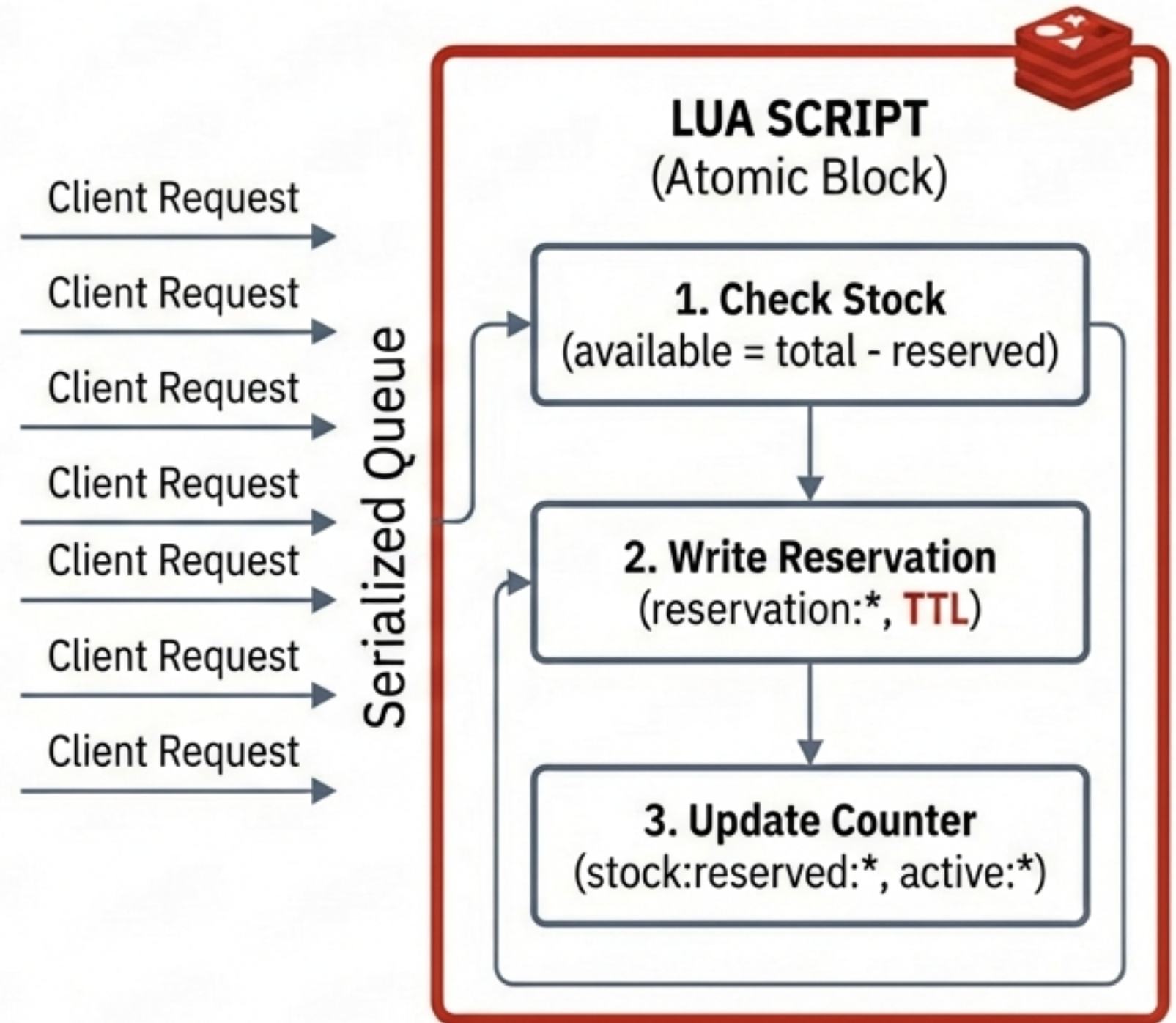
Objectif : initialiser `stock:product:<id>` et `stock:reserved:<id>=0`.

6.2 Réserveation Atomique (Lua)

Objectif: Empêcher la survente.

Mécanisme: Le script Lua effectue en une seule opération :

1. Vérification stock disponible (`available = total - reserved`).
2. Création de `reservation:*` avec TTL.
3. Incrément de `stock:reserved:*`.
4. Indexation dans `active:*`.



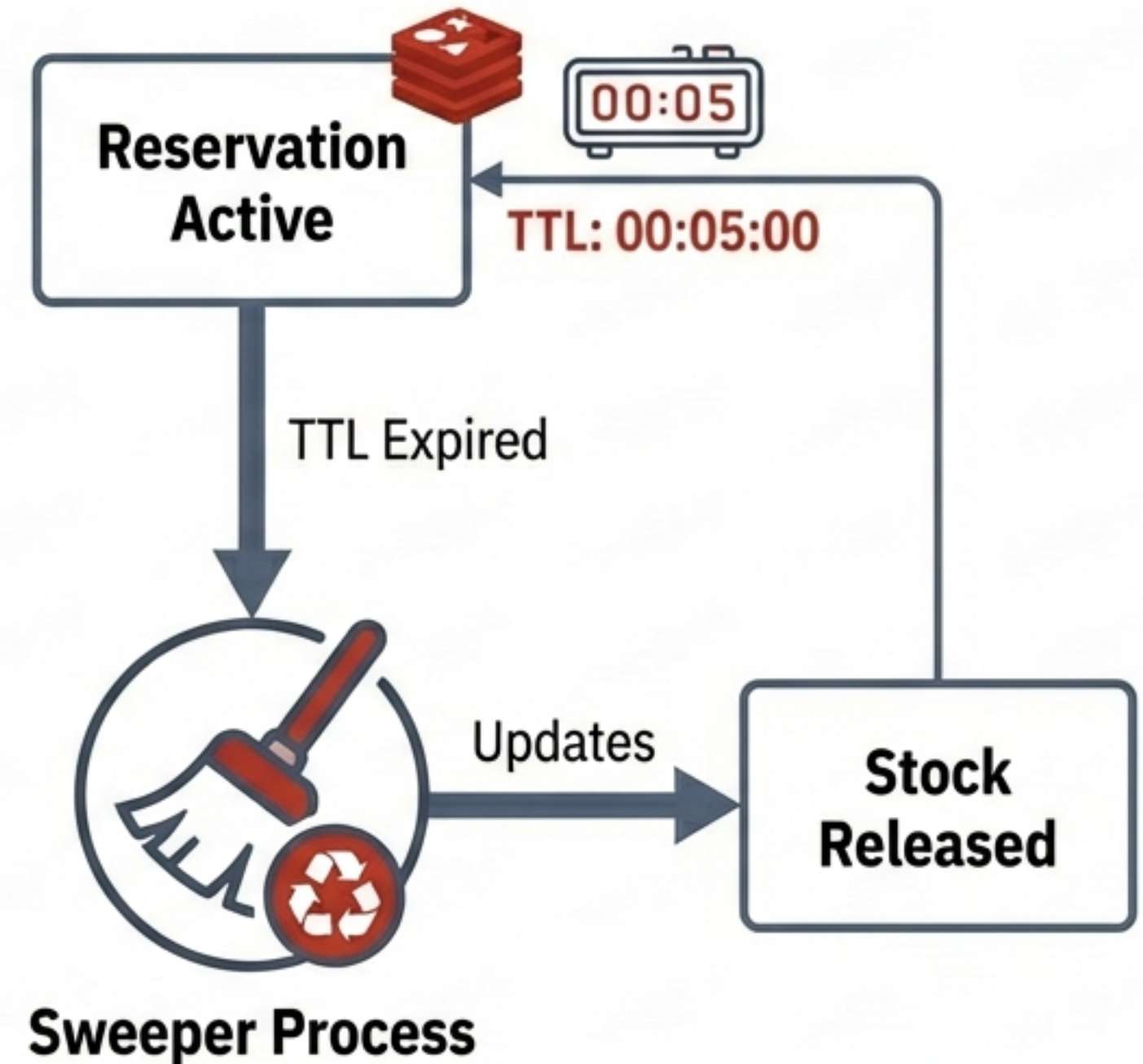
6. Implémentation : Cycle de Vie & Nettoyage

6.3 Annulation

Libérer immédiatement une réservation avant expiration.

6.4 Expiration et libération automatique (Sweeper)

- **Problème:** Si reservation:* expire (TTL), le stock doit être libéré.
- **Solution:** Le 'Sweeper' s'appuie sur :
 - `active:reservations` (Set) pour lister.
 - `active:reservation:qty` (Hash) pour connaître la quantité.
 - Mise à jour de `stock:reserved:\*` + audit.

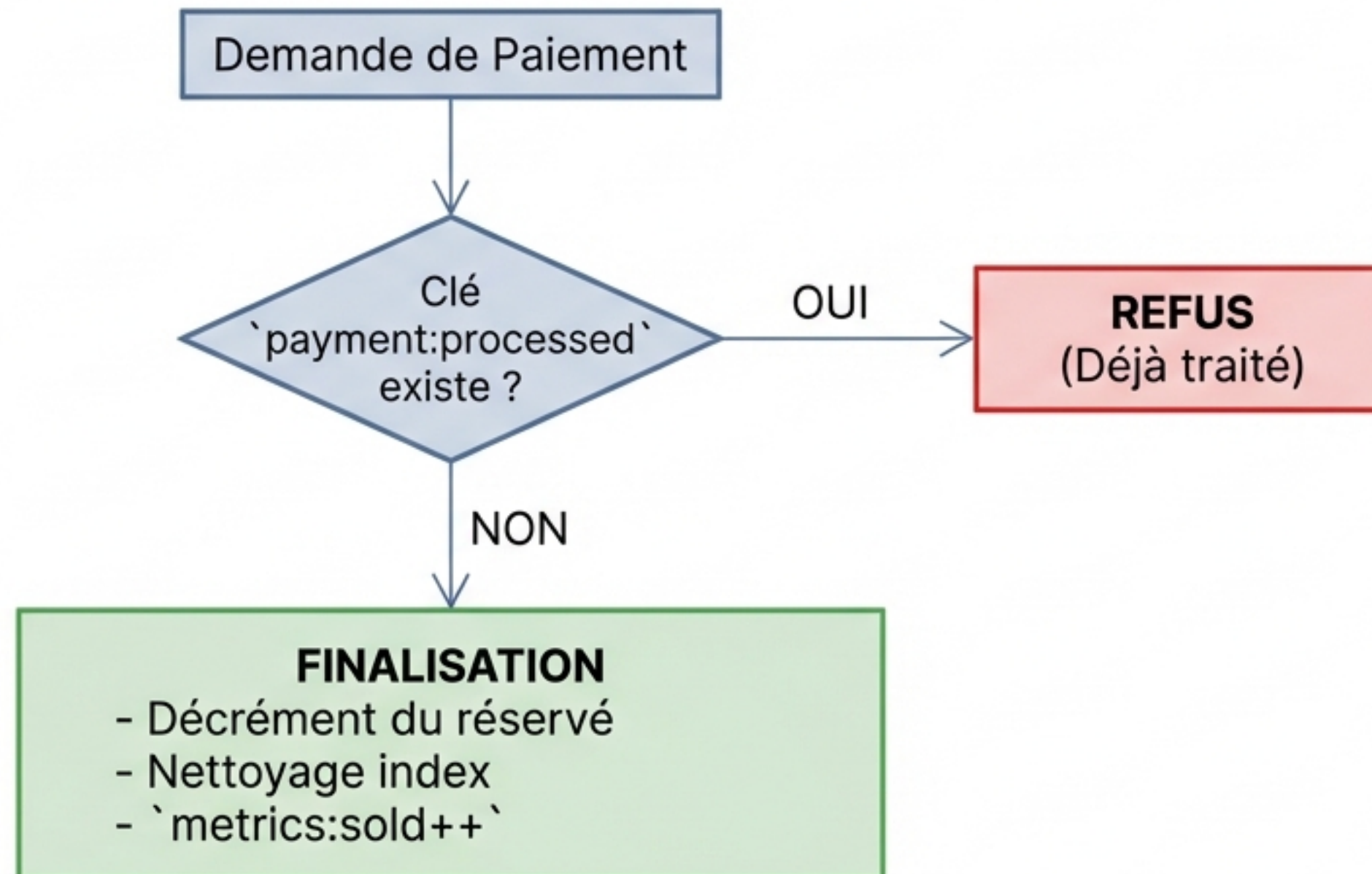


6. Implémentation : Confirmation & Idempotence

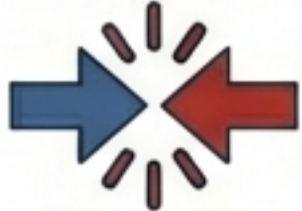
6.5 Paiement confirmé

Objectif: Empêcher le double paiement.

Technique: Création de la clé `payment:processed:<order>` avec `SET NX EX`.



7. Gestion des Cas Limites

	Risque	Solution Technique
	1. Survente (Race Condition)	1. Évitée via Lua (atomicité check+update).
	2. Double paiement	2. Évité via Idempotence (<code>`payment:processed`</code>).
	3. Requêtes répétées	3. Contrôles préalables (déjà réservé / déjà traité).
	4. Abandon tardif	4. TTL + Sweeper garantissent la libération.
	5. Données manquantes	5. Compteur <code>metrics:incidents</code> et trace <code>`audit:stock`</code> .

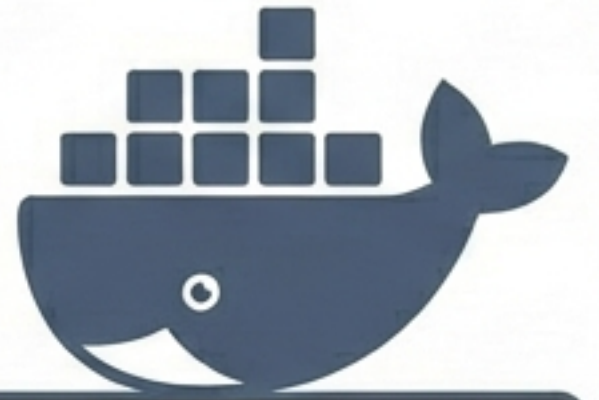
8. Installation & Environnement

8.1 Prérequis

- Docker Desktop
- Python 3.11+
- Bibliothèque Python Redis (`pip install -r requirements.txt`)

8.3 Configuration

- Variables d'environnement (PowerShell/ACL) :
``$env:REDIS_USERNAME``
``$env:REDIS_PASSWORD``
- Commande de santé :
``python app.py ping``



8.2 Démarrage (Docker)

```
> docker run -d --name redis -  
flash -p 6379:6379 redis:7
```

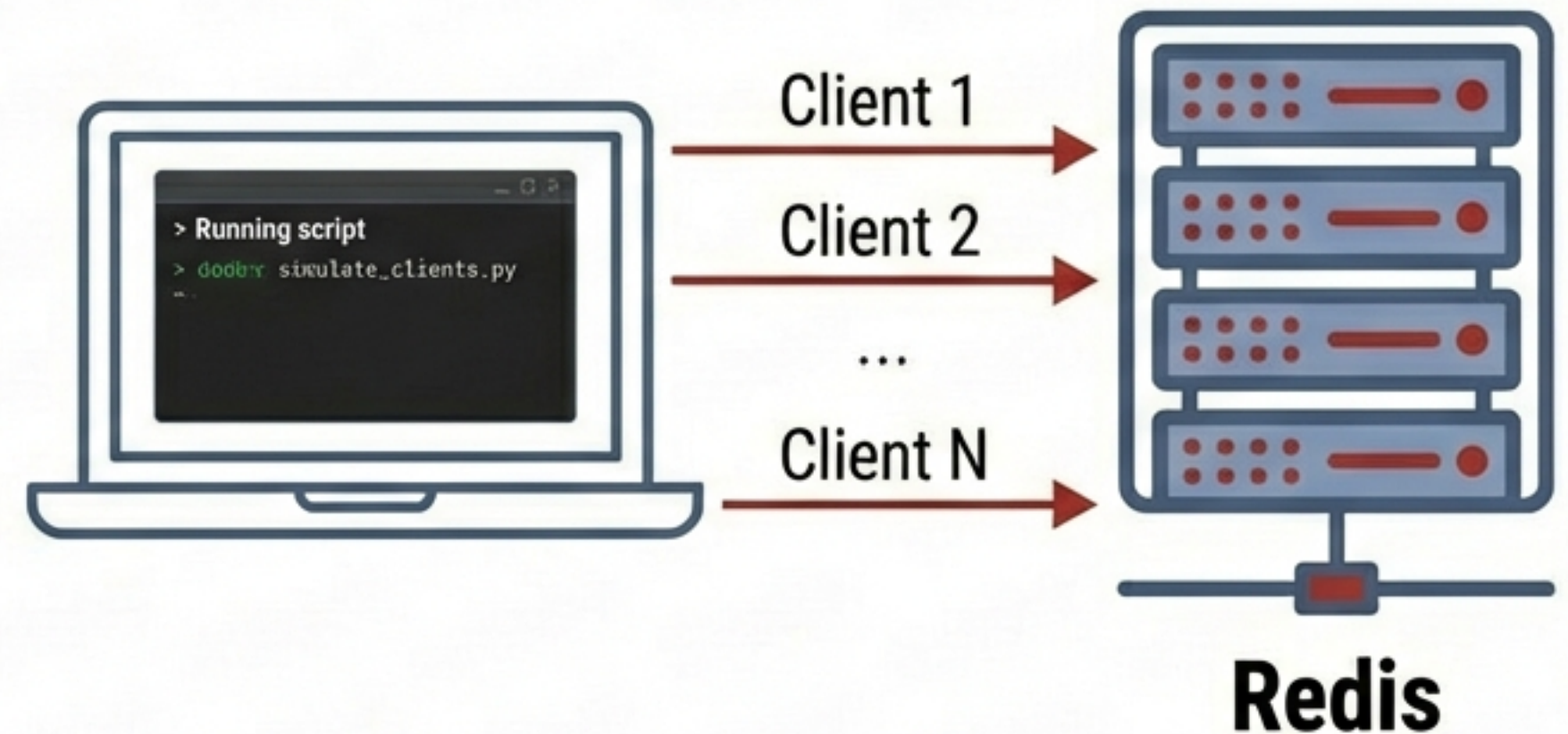

9. Simulation & Scénarios de Test

9.1 Simulation Multi-clients

- Script : `simulate_clients.py`
- But : Simuler N clients en parallèle (reserve/confirm/cancel/abandon).

9.2 Comparatif SAFE vs UNSAFE

- Script : `scenario_concurrency.py`
- But : Prouver expérimentalement la survente si on n'utilise pas l'atomicité.



Paramètres de Test

Product ID: 101 | Stock: 20 | Clients: 200 | TTL: 8s

9. Résultats & Preuves

Interprétation des résultats

- **SAFE (Lua)** : `reserved` ne dépasse jamais `total` -> **PASS**.
- **UNSAFE (Naïf)** : `reserved` > `total` ou `available` < 0 -> **FAIL** (Survente).
- **Logs** : Les résultats sont archivés dans `2_Simulation/results_logs.txt`.



10. Planification (Sprints)

Spaces / Flash Sale - Redis / 130-23

Epic 2 – Réserveation et Expiration du Stock

Description
En tant que client, je réserve un stock temporairement avec expiration automatique.

Child work items

Work	Priority	Assignees	Status
FSR-29 Documentstion du Sprint 2 et validation des résultats	Medium	Khalid Merjen	Done
FSR-24 Conception du modèle de clés Redis pour stock, réservation et panier	Medium	Khalid Merjen	Done
FSR-25 Définition des règles d'expiration des réservations	Medium	Khalid Merjen	Done
FSR-26 Simulation de plusieurs clients réservant simultanément	Medium	Anas Ighroul	Done
FSR-27 Gestion de l'expiration et libération automatique du stock	Medium	Anas Ighroul	Done
FSR-28 Implémentation de la réservation temporaire du stock avec TTL	Medium	Anas Ighroul	Done

150% Done

Spaces / Flash Sale - Redis / FSR-54

Epic 4 – Simulation de charge & preuve anti-survente

Description
Tester le système sous forte concurrence et démontrer que la solution empêche la survente.

Child work items

Work	Priority	Assignees	Status
FSR-63 Simulation multi-clients	M	A	Done
FSR-64 Comparaison SAFE vs UNSAFE	M	A	Done
FSR-65 Analyse des résultats	M	K	Done
FSR-66 Rapport & preuves	M	K	Done

100% Done

Sprint 1

Analyse métier,
modèle de clés.

Sprint 2

Réservation TTL +
Atomicité Lua.

Sprint 3

Paiement idempotent
+ métriques.

Sprint 4

Stabilisation, comparatif
SAFE vs UNSAFE.

11. Conclusion

Verdict

Ce projet démontre qu'un système de vente flash peut être sécurisé contre la survente.



Redis :
Performance et simplicité.



Lua :
Élimination des conditions de course (Atomicité).



TTL :
Gestion automatique de l'abandon panier.



Preuve :
La méthode atomique (SAFE) préserve la cohérence.

